Global Asmts & Rules

Global Assumptions and Rules

The following engineering decisions require explicit values before implementation. No implicit assumptions shall be made.

System Assumptions

- Simulation time step = @
- Timestamp format = @
- Time zone = @
- Historical data source = @
- Forecast horizon = @
- Forecast update frequency = @

Storage Assumptions

- Maximum charging power = @
- Charging efficiency = @
- Discharging efficiency = @
- Round-trip efficiency = @
- Minimum allowable state of charge = @
- Maximum allowable state of charge = total_capacity
- Self-discharge rate = @

Transmission Assumptions

- Shared transmission rules = @
- Maximum import/export assumptions = @
- Curtailment rules = @

Dispatch Rules

- Primary optimization objective = @
- Secondary optimization objective = @
- Dispatch priority hierarchy = @
- Recharge priority hierarchy = @
- Tie-breaking rules = @

Event Rules

- Minimum stress event duration = minimum_window
- Event merge rules = @
- Event split rules = @
- Overlapping event rules = @
- Event termination rule = @

Error Handling

- Missing data policy = @
- Duplicate timestamp policy = @

- Invalid input policy = @
- Numerical overflow policy = @
- Simulation recovery policy = @
- Logging policy = @
- Version tracking policy = @

Global Interf. Contract

Global Interface Contract

Every software component shall satisfy the following interface requirements.

Inputs
Every component shall receive:

- validated data only
- immutable historical datasets
- immutable scenario configuration
- mutable simulation state only through defined contracts

Outputs
Every component shall produce:

- deterministic outputs
- versioned outputs
- schema-compliant outputs
- validated outputs

Interface Rules
Software components shall never modify another component's output.

Software components shall never bypass an intermediate component.

Every component shall return exactly one of:
Success

- Recoverable Error
- Non-Recoverable Error
- Validation Failure

Each response shall include:

Field	Description
execution_status	@
execution_timestamp	@
software_component	@
execution_duration	@
warning_count	@
error_count	@
execution_message	@

Schema Versioning

Every exchanged object shall include:

Field	Description
schema_version	@
software_version	@
simulation_version	@
calculation_version	@
configuration_version	@

Gloval Data Governance

Data Governance

All software components shall conform to the following engineering rules.

Naming Convention

- identifier format = @
- timestamp format = @
- units convention = @
- null value representation = @
- missing value representation = @

Precision

- floating point precision = @
- numerical rounding policy = @
- currency precision = @
- timestamp precision = @

Storage

- persistent storage location = @
- temporary storage location = @
- serialization format = @
- compression policy = @
- archive policy = @

Logging

Each component shall record:

- execution start
- execution finish
- warning messages
- validation failures
- execution duration
- software version
- configuration version
- schema version
- @

1-Scenario Config

Software Component 1 — Scenario Configuration

Purpose
Collect, validate, and store all user-defined scenario parameters before simulation begins.

Data Fields
Validation Rules

- All inputs shall be validated before simulation begins.
- Invalid inputs shall terminate execution before data loading.
- Validation ranges:
    - wind_generation_multiplier = @
    - transmission_capacity = @
    - total_capacity = @
    - reserve_power_output = @
    - minimum_window ≥ @

Data Contract

Field	Required	Type	Default	Unit	Description
scenario_id	Y	str		none	Unique scenario identifier
wind_generation_multiplier	Y	float	1.0	Scalar	Historical wind multiplier
transmission_capacity	Y	float	200.0	MWh	Maximum transmission energy capacity
total_energy_capcity	Y	float	60,000.0	MWh	Maximum energy capacity for storage
total_storage_units	Y	int	3000	Storage Units	
reserve_power_output	Y	float	4320.0	MW	Maximum storage power output
minimum_event_window	Y	int	2	Days	Minimum stress event duration

2-Data Pipeline

Software Component 2 — Data Pipeline

Purpose
Extract, transform, validate, and prepare historical data for simulation.

Data FieldsProcessing Steps
For each historical winter:

- Winter 2021/2022
- Winter 2022/2023
- Winter 2023/2024
- Winter 2024/2025
- Winter 2025/2026

perform the following sequence independently.

Extract Data
Load:

- hourly electrical load
- hourly wind generation
- hourly oil generation
- hourly gas generation
- transmission information

Transform Data
Apply:
historical_wind_generation
× wind_generation_multiplier
= scaled_wind_generation

Validate Data
Verify:

- no missing timestamps
- no duplicate timestamps

- chronological ordering
- valid numerical ranges
- @additional validation rules

Data Contract

Field	Required	Type	Unit	Description
date	Y	YYYY-MM-DD	date	Calendar date
hour	Y	int	hour	Hours in a day
load_mw	Y	float	MW	Maximum transmission energy capacity
wind_mwh	N	float	MWh	Hourly wind generation
oil_mwh	N	float	MWh	Hourly oil generation
gas_mwh	N	float	MWh	Hourly liquid natural gas generation
demand_percentile	N	float, constant per date	%	Daily? historical percentile of demand load
wind_forecast_frac	N	float, constant per date	ratio	Daily? wind generation fraction of total 7-day wind generation

Optional Fields

- @

Validation Rules

- continuous timestamps
- chronological ordering
- no duplicate timestamps
- no null required values
- valid numerical ranges
- @

3-Stress Event Detection

Software Component 3 — Stress Event Detection
Purpose

Identify supply adequacy stress events requiring storage simulation.

Event Detection Rule

Calculate:
historical_daily_load

Determine:
90th historical daily load percentile

A stress event begins when:
daily_load ≥ 90th percentile
for
minimum_window
consecutive days.

Store:

- event identifier
- starting timestamp
- ending timestamp
- duration

Data Contract

Field	Required	Type	Unit	Description
event_id	@	@	@	@
winter_id	@	@	@	@
event_start_timestamp	@	@	@	@
event_end_timestamp	@	@	@	@
first_hour_index	@	int	hour	@
last_hour_index	@	int	hour	@
event_duration	@	int	day	@
peak_daily_load	@	tuple?	MW?	@
percentile_threshold	@	int	Percentile	@

Derived Fields

- event hour count
- event day count
- @

Validation Rules

- duration ≥ minimum_window
- start precedes end
- event entirely contained within historical winter
- no overlapping event identifiers
- @

4-Simulation Initialization

Software Component 4 — Simulation Initialization

Purpose
Initialize the storage system before each stress event simulation.

Initial Event
Beginning State of Charge
SOC = 0 MWh

Charging Window
Maximum available charging period:
5 days
representing:
7-day forecast
− minimum_window

Subsequent Events
Beginning State of Charge
SOC
= remaining storage from previous event

Initialize:

- storage state
- dispatch history
- recharge history

5-Storage Dispatch Engine

Software Component 5 — Storage Dispatch Engine

Purpose
Determine hourly storage operation during each stress event.

Begin Rolling Forecast Simulation
For each hour between:
event_start
and
event_end
perform:
Evaluate Current System State
Determine:

- available storage energy
- remaining energy capacity
- maximum power output
- available transmission capacity
- @additional operating constraints
- Build Discharge Schedule

Construct an hourly discharge schedule using:

- available storage
- reserve limits
- transmission limits
- @dispatch priorities

Estimate Future Recharge Opportunities
Estimate available recharge throughout the remaining forecast horizon.

Predict Storage Trajectory
Forecast:
future SOC

throughout the remaining event.

Determine Current Dispatch
Calculate:
capacity_dispatched(t)
subject to:

- storage energy capacity
- reserve power output
- transmission limits
- minimum reserve
- @charging constraints
- @round-trip efficiency
- @dispatch objective
- @additional operating rules

6-Grid Simulation Engine

Software Component 6 — Grid Simulation Engine

Purpose
Calculate hourly grid behavior after storage dispatch.
For every simulation hour calculate:

- observed net load
- dispatched net load
- capacity dispatched
- updated storage state
- remaining storage
- @additional grid variables

Update:
SOC(t+1)
Confirm:

- stress event continues
- event completed

Advance simulation hour.

7-Scenario Metrics Engine

Software Component 7 — Scenario Metrics Engine

Purpose
Calculate scenario performance metrics after each winter simulation.

Capacity Margin Improvement

$$\sum_{t=0}^{N} (net\_load\_dispatch(t) - net\_load\_observed(t)) \bullet 100\%$$

Stress Window Effectiveness

$$\sum_{t=0}^{N} (capacity\_dispatched(t)) \,/\, \sum_{t=0}^{N} (oil\_generation\_actual(t) + gas\_generation\_actual(t) + wind\_generation\_actual(t))$$

Fuel-Fired Generation Offset

$$\sum_{t=0}^{N} (oil\_generation(t)) + \sum_{t=0}^{N} (gas\_generation(t)) - (\sum_{t=0}^{N} (wind\_generation(t) + \sum_{t=0}^{N} capacity\_dispatched(t))$$

Fuel Offset Percentage

$$fuelfired\_generation\_offset / \sum_{t=0}^{N} total\_generation(t)$$

Cycle Recharge Mismatch

$$\sum_{t=t_{start}}^{t_{end}} (recharge\_opporunity(t) - actual\_recharged(t))$$

Average Recharge Mismatch

$$\frac{1}{N} \sum_{c=1}^{N} cycle\_recharge\_mismatch(t)$$

Recharge Capacity Mismatch
$average\_cycle\_recharge\_utilization(t)$ / $maximum\_available\_capacity$

Estimated Capital Costs
$est\_transmission\_cost\_per\_mile \bullet miles \ + \ est\_storage\_unit\_cost \bullet total\_unit\_count$

Cost per Equivalent Full Cycle
$est\_capital\_costs$ / $((annual\_equivalent\_full\_cycle) \bullet (solution\_lifetime))$

Scenario Robustness Score

Each winter shall be evaluated independently.

Winter evaluations:

2021/2022
2022/2023
2023/2024
2024/2025
2025/2026

Evaluation Rule:

Each performance metric shall define:

acceptable operating range = @
warning range = @
failure threshold = @

If any single metric falls outside its acceptable operating range during a winter simulation, that winter automatically receives one robustness failure point.

The final Scenario Robustness Score shall be calculated
from the total number of winter failure points
using: @

Archived

V1.0 MVP Architecture

1. Gather user inputs
    a. wind_generative_scale
    b. transmission_capacity
    c. total_capacity
    d. minimum_window
    e. reserve_power_output
2. Begin 5 year loop
3. Extract, transform, and load data for current year's daily loads
4. Search for supply adequacy stress events in the current year by daily loads
    a. Store events by indexes for the first and last hour of the event
5. Initialize the simulation
    a. Calculate state of charge for the beginning of day 0 event dispatch
        i. The first event
            1. Starts with 0 MWhs of energy
            2. Has a 5 day maximum number of days for storing energy before the minimum window—representing the shortest 2-day window in a 7-day forecast
        ii. i+1 Events
            1. Starts with X MWhs of energy, where X is the charge remaining at the end of the last event
            2. Same 5 day charge maximum
    b. Define system state
6. Begin a 7-day rolling-window analysis that loops over the starting and ending index to each event
    a. Evaluate today's available energy capacity
    b. Build discharge schedule
    c. Estimate future recharge opportunities
    d. Predict future storage trajectory
7. Determine today's dispatch strategy
8. Calculate grid performance during simulation
    a. Capacity margin: total load - net load with storage
9. Return the remaining storage after grid simulation
10. Update system state
11. Confirm the stress window hasn't ended
12. Advance the rolling-window loop
13. Summarize scenario results
    a. Capacity Margin Improvement
    b. Stress Window Effectiveness
    c. Fuel Offset Percentage
    d. Recharge Capacity Mismatch
    e. Estimated Capital Costs
    f. Cost per Equivalent Full Cycle
    g. Scenario Robustness Score

14. Build an annotation decision package
15. Send the decision package to the AI assistant

Step 1 - User Inputs

- Pumped Hydro Storage
    - Total Capacity
    - Starting Capacity
    - Power Output
- Season
    - Calendar date range
- Minimum stress event window in days
    - Severity stress level based on a percentile of historical load records
- Transmission limits

Step 2 - Search for stress event in calendar date range

- Calculate daily load as a sum of hourly load in MWh across everyday in time range
- Run a search function over daily loads in that time range looking for windows of X or more stress days (where X is defined by the user)
- Store all windows found

Step 3 - Calculate state of charge for the beginning of the first stress day

- This will frequently be 100%, but both fewer days and lower starting capacity can decrease capacity at the beginning of an event
- Check ∑wind_power(d) for (d=0,N) is greater than soc_max - soc_start
    - soc_window_a = soc_max - soc_start
    - Else:
        - soc_window_a += ∑wind_power(d)

Step 4 - Confirm stress event starting values with the user

- Return to the user the total number of days before the stress event occurs and the state of charge on the first stress day
- List state of charge options for each day the event is moved closer to the first stress day, starting with the first calendar date
- Ask the user what is the first day they'd like to base calculations on

Step 5 - Estimates the daily energy budget for each day

- For hours between the last discharge hour of the previous stress day and the first discharge hour of today
    - recoverable_energy = min(ForecastWindEnergycharge, max_charge • (t_2 - t_1))
    - required_recharge = capacitymax - socfloor - strategic_reserve
    - recharge_sufficiency_ratio = recoverable_energy / required_recharge
    - Check for days with a recharge sufficiency ratio less than 1
        - last_day_below = day_index
-

Step 6 -

-

Step 7 -

- Recursively call the usable energy
    - t starts at 0
    - usable_energy(t) = soc(t) - soc_floor - strategic_reserve

        - usable_energy is the total energy available to discharge at the start of every hour
        - soc is the state of charge or the current amount of energy stored in pumped hydro reserves
        - soc_floor is a percentage of total energy capacity the system isn't typically allowed to discharge
        - strategic_reserve is a percentage of total energy capacity the system holds in reserve on top of the state of charge floor, which can be used during early warning signs of system stress and increases optionality
    - Check to see if the current hour coupling is within the peak 2 hour coupling
        - $NetLoad(t)=Load(t)-Solar(t)-Wind(t)$
        - $PeakWeight(t) = NetLoad(t) / \sum NetLoad_{peak}$
        - $Discharge_{peak}(t) = E_{peak} \times PeakWeight(t)$
    - Check to see if current hour is within 4 hours before or after the peak 2 hour coupling
        - $Ramp(t) = |NetLoad(t)-NetLoad(t-1)|$
        - $SmoothWeight(t) = Ramp(t) / \sum Ramp_{window}$
        - $Discharge_{smooth}(t) = E_{smooth} \times SmoothWeight(t)$
    - Else: check to see if current hour falls outside the first two conditions
        - Check wind_power(t) is greater than soc_max - soc(t)
            - charge(t)=soc_max - soc(t)
            - Else:
                - charge(t)=wind_power(t)
        - soc(t+1) = soc(t)+charge(t)
            - charge is equal to the total historical wind power generated in that hour
    - Increase the sum of total days of total_usable_energy within the stress window by the usable_energy of the current hour
        - total_usable_energy(d) += usable_energy(t)+charge(t)
- Calculate what 80% of the total energy budget for the stress window
    - Sum of wind power forecasts
    - The difference of today's state of charge minus constant reserve of 33% total capacity
    - Sum of the previous two calculations for a total energy budget
    - Product of total energy budget times 0.80
    - A simple average of 80% energy budget divided by number of days across the window

Step 6 -

Available Supply(d) = Available Generation(d) + Imports(d)
Capacity Margin(t) = Available Supply(d) - Load(t)

Peak Reduction Allocation
NetLoad(t)=Load(t)−Solar(t)−Wind(t)
PeakWeight(t) = NetLoad(t) / ∑NetLoadpeak
Dischargepeak (t) = Epeak × PeakWeight(t)

Net Load Smoothing Allocation
Ramp(t) = |NetLoad(t)−NetLoad(t−1)|
SmoothWeight(t) = Ramp(t) / ∑Rampwindow
Dischargesmooth(t) = Esmooth × SmoothWeight(t)

ResidualLoad(t)=Load(t)−Wind(t)−Storage(t)
Cost(t)=ResidualLoad(t)×Costgas

Total Dispatch
Discharge(t) = Dischargepeak(t) + Dischargesmooth(t)
∑Discharge(t) ≤ Budget(d)

State of Charge
soc_now >= .33 soc_total + strategic_reserve
soc(t+1) = soc(t)+charge(t) • efficiency - (discharge(t) / efficiency)
usable_energy(t) = soc(t) - soc_floor - strategic_reserve
total_usable_energy=usable_energy(0)+∑ charge(t) for (0,N)
DemandPercentile(d) = Demand(d)/(561,878MWh for summer or 434,214MWh for winter)
FutureWind(d)=∑ WindForecast(i) for (i=d+1, N)
max(FutureWind)=∑ WindForecast(i) for (i=d, N)
WindForecast(d)=FutureWind(d)/max(FutureWind)
Priority(d) = 0.7 DemandPercentile(d) + 0.3 WindForecast(d)
Usable_Energy(d)=∑Usable_Energy(t) for (t=0, 24)

recoverable_wind_energy=min(ForecastWindEnergycharge, Pcharge,max×Tcharge)
required_recharge_energy=capacitymax - socfloor - strategic_reserve

Daily Energy Budget
Budget(d) = Usable_Energy(d) x (Priority(d)/∑Priority)